

Wireless design file converter

You watched this converter on Day 1. Today you design it, build it, and put it on GitHub with a check that runs on every push. Yours goes both ways.

SET UP

Open the folder. In the desktop app, open the Code screen, start a new session and choose the converter folder inside the class folder on your Desktop, the one you downloaded before class. It holds three files:

- `esx2oi.py`, a published script that turns an Ekahau `.esx` into an Open Intent zip
- `LICENSE`, the script's licence
- `ekahau_sample.esx`, a real design: one floor, 446 walls, 212 access points

Check what loaded. Type `/context` and press return. Two things should be there: your `~/ .claude/CLAUDE.md` under the memory files, and superpowers' skills in the skills list (brainstorming, writing-plans and the rest). If either is missing, yesterday's lab didn't stick: ask.

Pick the model. Type `/model`, press return, and pick Sonnet from the list. It does this build well and uses less of your plan than the default.

TASK 1: DESIGN IT

The page gets designed before it gets built: a look you like, a brief, a design, then the hand-off to Code.

Find a look. Open `styles.refero.design` or `mobbin.com` in your browser and find a page you'd like yours to feel like: a simple upload page, a file converter, a drop zone. Take a screenshot of it.

Brief and design. In the desktop app, click Design in the sidebar, then the Design tile under "Make something new". Paste the screenshot into the message box and type:

```
This is the look I want. The page is a wireless design file converter: you drop an Ekahau .esx or an Open Intent zip on it and get the converted file back as a download. It works out which way to convert. A file that's neither gets a plain message. Nothing is kept. Write me the design brief for this page first, then design it.
```

Read the brief it writes and strike anything you wouldn't say. Click through the design. Change it once in the chat and once by hand on the canvas.

Hand it to Code. Open the design's export menu and choose the hand-off to Claude Code. It lands in your Code session.

TASK 2: BUILD IT

What the converter has to do:

- Turn an Ekahau .esx into an Open Intent zip. The supplied `esx2oi.py` already does this.
- Turn an Open Intent zip back into an .esx. This doesn't exist yet.
- Prove the round trip: `ekahau_sample.esx` to Open Intent, back to .esx, forward once more, and the second Open Intent holds the same floor plan, walls, wall materials, access points and image as the first. A check you can run.
- Run as a web page on your laptop, built to the design from Task 1: drop a file, get the converted file back as a download. It works out which way to convert. A file that's neither gets a plain message, not a crash. Nothing is kept: no accounts, no history, no files left behind.

Write the prompt yourself, in your own words, from the list above. Say that the design was handed over from Claude Design, and open with the two things you set up yesterday:

```
Use superpowers and my build instructions. Brainstorm it with me before you plan anything.
```

Let the pipeline run. It asks, you answer; it offers approaches, you pick; it shows a plan, you read it; it builds in phases, you look at each one running.

Expect it to break. It will not work first time; nothing real does. A check fails, the page won't start, the file that comes back is wrong. That is the normal state of a build, and fixing it is Claude's job, not yours: tell it what you saw, in plain words ("the page says 500 when I drop the sample", "the round trip puts every wall on the wrong side"), and let it find the cause. Describe the symptom; don't guess the fix. If you're going round in circles, say so and ask it to step back and explain what it thinks is happening.

Check it. Run the round-trip check yourself, then put `ekahau_sample.esx` through the page and open what comes back. Then break it on purpose: drop a file that's neither, and see what the page says.

TASK 3: PUSH IT, WITH CI

The snake went to GitHub Pages and got a web address. This one won't. Pages serves files; the browser ran your snake itself. The converter is Python, and Python needs a machine to run on, which Pages doesn't have. So the repo holds the code, the workflow runs your check on every push, and the app runs on your laptop. Putting it on a public server is a different job for another day.

A README. Have Claude write a short one that says what it is and how to run it.

A repo. A new GitHub repo of your own, the folder pushed to it, the way you did in push-to-live.

A workflow. A GitHub Actions workflow that runs your round-trip check on every push. The first run goes green on the Actions tab.

YOU'RE DONE WHEN

- The round-trip check passes.
- `ekahau_sample.esx` goes through your page and comes back as Open Intent, and that Open Intent goes through your page and comes back as an `.esx`.
- Your repo is on GitHub with a green run on the Actions tab.
- You've shown Rob the round trip.

If you have Ekahau or Hamina on your laptop, open your files in them.